

Rules of theoretical exam on Algorithms and Data Structures.
MIPT. MSAI 2025.
~ January 2026

Before the exam, bot will assign a time slot (1 hour) for each student (we will discuss convenient time slots with you in advance). During this hour you will have 1-to-1 zoom call with the teacher and answer exam questions.

1 hour before you time, bot will randomly choose 2 questions from the list below for you. You should prepare a short answer for given questions (answer should take ~15-20 minutes). You may use any materials while preparing an answer.

During the assigned time slot you present your answer to the teacher (you can use handwriting and send me photos in telegram, or use touchscreen to draw, or prepare a document, whatever, but please, make your materials presentable).

After presenting your answer, you may be given several additional questions. Additional questions may exceed the list of questions below, but they do not exceed the program of the course (e.g. you may be suggested to solve some simple problem orally).

In total you will have 1 hour for preparation and your answer will take no longer than 1 hour.

N.B.: When you choose the time slot for the exam, you are choosing the time when you start your answer. You should also have ~ 1 hour before this time to prepare answers for the questions.

List of questions:

1. Algorithm complexity basics. Worst case complexity. Amortized complexity (prove for vector).
2. Basic data structures. Stack, Queue, Vector, Linked List, Doubly Linked List.
3. $O(N^2)$ sorting algorithms. Counting sort. Proof that sorting by pairwise comparisons faster than $O(N \log N)$ is impossible.
4. Linearithmic sorting algorithms: Merge Sort, Quick sort, Heap sort (without explaining how heap works).
5. Binary search (item in a sorted array; left/right binary search; binary search on a function; binary search by answer).
6. Binary heap (invariant; SiftUp/SiftDown; how to store heap in vector (python list); push/pop/remove; building heap).
7. Basic principles of Dynamic Programming. Fibonacci numbers: naive solution, top-down, bottom-up approach. Longest Increasing Subsequence (LIS): $O(N^2)$, $O(N \log N)$.
8. Knapsack problem: (problem statement; DP solution; answer restoration; modifications).
9. Ways to store graphs. DFS (connected components search; cycles search; topological sort). BFS (shortest paths).
10. Knuth Morris Pratt (KMP) algorithm.
11. Dijkstra's algorithm (prove that algorithm returns shortest paths)
12. Bellman-Ford algorithm (how to detect negative cycles).
13. Floyd-Warshall algorithm (prove that we don't need to store solutions for 3-dimensional DP).
14. RMQ/RSQ (prefix sum; sparse table; sqrt-decomposition)
15. RMQ/RSQ: (fenwick tree; segment tree)
16. Hash (prove expectation of number of collisions to be N/M for random function; universal families). Polynomial hash for strings.
17. Rabin-Karp algorithm.
18. Hash table (separate chaining collision handling; open addressing collision handling).